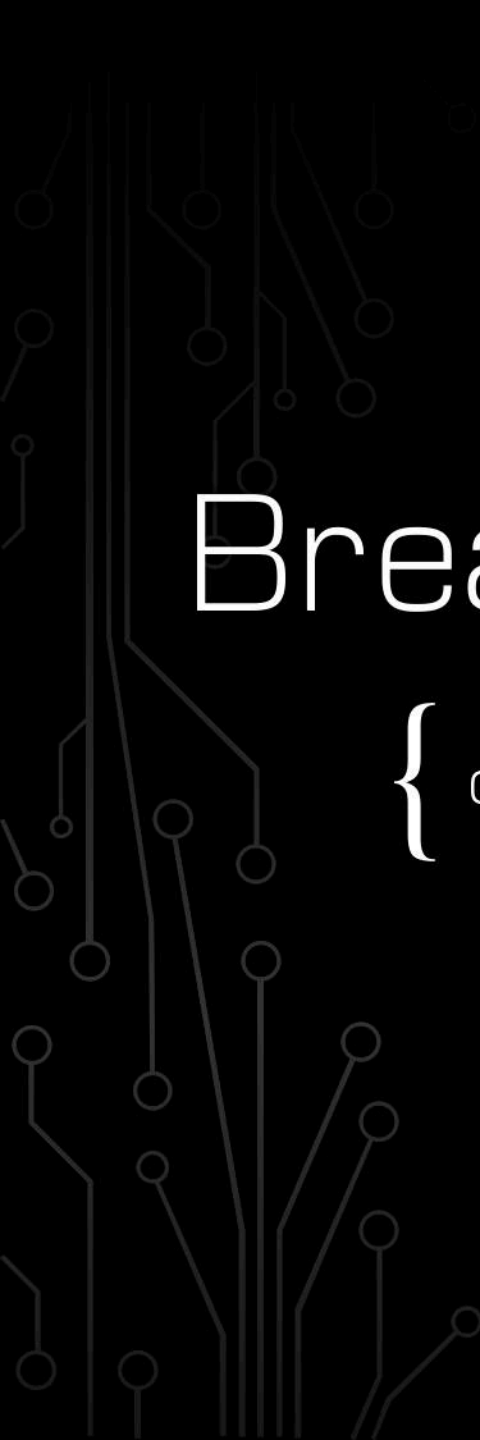




Breaking the x86 ISA

{ domas / @xoreaxeaxeax / DEF CON 2017



& Christopher Domas

✂ Cyber Security Researcher @
Battelle Memorial Institute

./bio

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& 8086: 1978

& A long, tortured history...

The x86 ISA



& Modes:

- ⌘ Real (Unreal)
- ⌘ Protected mode (Virtual 8086, SMM)
- ⌘ Long mode (Compatibility, PAE)

x86: evolution

A decorative graphic on the left side of the slide, consisting of a vertical line with various horizontal and diagonal branches, ending in small circles, resembling a circuit board or a stylized tree.

& Instruction sets

x86: evolution

- 
- ⌘ Modern x86 chips are a complex labyrinth of new and ancient technologies.
 - ⌘ Things get lost...
 - ⌘ 8086: 29,000 transistors
 - ⌘ Pentium: 3,000,000 transistors
 - ⌘ Braodwell: 3,200,000,000 transistors

x86: evolution

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& We don't trust software.

- ⌘ We audit it
- ⌘ We reverse it
- ⌘ We break it
- ⌘ We sandbox it

Trust.

- 
- A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.
- ⌘ But the processor itself?
 - ⌘ We blindly trust

Trust.



& *Why?*

& Hardware has
all the same problems as software

& Secret functionality?

⌘ Appendix H.

& Bugs?

⌘ F00F, TSX, Hyperthreading.

& Vulnerabilities?

⌘ SYSRET, cache poisoning, sinkhole

Trust.

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram. The lines and circles are arranged in a vertical, branching pattern, with some circles acting as nodes or junctions.

& We should stop
blindly trusting our hardware.

Trust.

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram. The lines are vertical and horizontal, with some diagonal connections, and the circles are of varying sizes, some acting as nodes or junctions.

& What do we need to worry about?



& Well known from software
⌘ Examples

Backdoors



& Hardware

- ⌘ FPGAs
- ⌘ Hypervisors
- ⌘ Microcode
- ⌘ Supply chain

Backdoors

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& Could a hidden instruction
unlock your CPU?

Hidden instructions



& Historical examples

⌘ ICEBP

⌘ apicall

Hidden instructions

Table A-2. One-byte Opcode Map: (00H – F7H) *

	0	1	2	3	4	5	6	7
0	Es, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	rAX, Iz	PUSH ES ³⁴	POP ES ³⁴
1	Es, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	rAX, Iz	PUSH SS ³⁴	POP SS ³⁴
2	Es, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	rAX, Iz	SEG=ES (Prefix)	DAA ³⁴
3	Es, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	rAX, Iz	SEG=SS (Prefix)	AAA ³⁴
4	eAX REX	eCX REX B	eDX REX X	eBX REX XB	eSP REX R	eBP REX RB	eSI REX RX	eDI REX RXB
5	rAX:r8	rCX:r9	rDX:r10	rBX:r11	rSP:r12	rBP:r13	rSI:r14	rDI:r15
6	PUSHA ³⁴ PUSHAD ³⁴	POPA ³⁴ POPAD ³⁴	BOUND ³⁴ Gv, Ma	ARPL ³⁴ Ew, Gw MOVSD ³⁴ Gv, Ev	SEG=FS (Prefix)	SEG=GS (Prefix)	Operand Size (Prefix)	Address Size (Prefix)
7	Jcc ³⁴ , Jb - Short-displacement jump on condition							
	O	NO	BN/AE/C	NB/AE/NC	ZE	NZ/NE	BE/NA	NBE/A
8	Eb, Ib	Immediate Gp 1 ^{1A} Ex, Iz	Eb, e ³⁴	Ex, Ib	Es, Gb	Ex, Gv	Ex, Gb	Ex, Gv
9	NOP PAUSE(F3) XCHG r8, rAX	rCX:r9	rDX:r10	rBX:r11	rSP:r12	rBP:r13	rSI:r14	rDI:r15
A	AL, Gb	rAX, Ov	Gb, AL	Gv, rAX	MOVB/B Yb, Xb	MOVSW/D/Q Yx, Xx	CMPS/B Xb, Yb	CMPS/W/D Xx, Yx
B	AL, Rb, Ib	CL, Rb, Ib	DL, Rb, Ib	BL, Rb, Ib	HL, Rb, Ib	HL, Rb, Ib	HL, Rb, Ib	HL, Rb, Ib
C	Eb, Ib	Ex, Ib	near RET ³⁴ Iw	near RET ³⁴ Iw	LES ³⁴ Gz, Mp VEX+2byte	LDS ³⁴ Gz, Mp VEX+1byte	Gep 1 ^{1A} + MOV	Gep 1 ^{1A} + MOV
D	Eb, 1	Ex, 1	Eb, CL	Ex, CL	AAM ³⁴ Ib	AAD ³⁴ Ib		XLAT XLATB
E	LOOPNE ³⁴ LOOPNZ ³⁴	LOOPE ³⁴ LOOPZ ³⁴	LOOP ³⁴ Jb	JNCZ ³⁴ Jb	AL, Ib	rAX, Ib	OUT Ib, AL	OUT Ib, eAX
F	LOCK Prefix		REP/REP XACQUIRE Prefix	REP/REP XRELEASE Prefix	HLT	CMC	Unary Gp 3 ^{1A} Eb	Unary Gp 3 ^{1A} Ev

Hidden instructions



& Traditional approaches:

- ⌘ Leaked documentation
- ⌘ Reverse engineering software
- ⌘ NDA

& But what if it's something stealthy?

Hidden instructions

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& Find out what's really there

Goal: Audit the Processor

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram. The lines are vertical and horizontal, with some diagonal connections, and the circles are of varying sizes, some acting as nodes or junctions.

& How to find hidden instructions?

Approach

& Instructions can be one byte ...

⌘ inc eax

⌘ 40

& ... or 15 bytes ...

⌘ lock add qword cs:[eax + 4 * eax + 07e06df23h], 0efcdab89h

⌘ 2e 67 f0 48 818480 23df067e 89abcdef

& Somewhere on the order of

1,329,227,995,784,915,872,903,807,060,280,344,576
possible instructions

Approach

- 
- ⌘ The obvious approaches don't work:
 - ✗ Try them all?
 - ✗ Only works for RISC
 - ✗ Try random instructions?
 - ✗ Exceptionally poor coverage
 - ✗ Guided based on documentation?
 - ✗ Documentation can't be trusted
[that's the point]
 - ✗ Poor coverage of gaps in the search space

Approach

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& A depth-first-search algorithm
& [Overview]

Tunneling

- & Catch: requires knowing the instruction length
- & Simple approach: trap flag
 - ✗ Fails to resolve the length of faulting instructions
 - ✗ Necessary to search privileged instructions:
 - ✗ ring 0 only: `mov cr0, eax`
 - ✗ ring -1 only: `vmenter`
 - ✗ ring -2 only: `rsm`
 - ✗ It's hard to even auto-generate a successfully executing ring 3 instruction:
 - ✗ `mov eax, [random_number]`
- & Solution: page fault analysis

Instruction lengths

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& (Overview)

Page Fault Analysis



& Trap flag

- ⌘ Catch branching instructions
- ⌘ Differentiate between fault types

Cleanup

- 
- ⌘ Reduces search space from 1.3×10^{36} instructions to $\sim 100,000,000$ (one day of scanning)
 - ⌘ This gives us a way to *search* the instructions space.
 - ⌘ How do we make sense of the instructions we execute?

Tunneling

- 
- A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.
- & We need a “ground truth”
 - & Use a disassembler
 - ⌘ It was written based on the documentation
 - ⌘ Capstone

Sifting



& Compare:

- ⌘ Observed length of instruction vs.
disassembled length of instruction
- ⌘ Signal generated by instruction vs.
expected signal

Sifting

```

164 r      shl ebx, 0x6b
      (unk)
      and edx, esi
      imul edx, dword ptr [rbx], 0x58112d43
s      movabs dword ptr [0x82d917b0fbb1eb5b], eax
a      push rsp
n      (unk)
d      or eax, 0x13753778
      ftst
v: 1      jbe 0xffffffffffffb9
l: 2      jle 0xffffffffffffdb
s: 5      and esi, esp
c: 2      and byte ptr [rax], al
      push -0x33da2f5b
      in eax, dx
l      mov esi, 0xe44908d6
f      pop rsp
t      mov eax, dword ptr [rdi + rax*4 - 0x2f5561f1]
e      (unk)
r      and 0x0000000000000000

```

```

c1e36b548611b50c1867138b8c9313 0
9a8c42843b3e09ee955b8d47d3869fd7 0
23d6190e7736d4e05487c839901e18ee :
6913432d1158e0a5c3a58f134f05d650 0
a35bebb1fbb017d982e12e7c7f3d833 1
540b58be4c5560f7fb0c26fad2ebc112 :
1e17200ec14fbac1c016d3116ca7847b 1
0d703775132ea748e46ab9f0182f2d2e 1
d9e47e167779d7867a13a560347ee110 .
76b7b83518xexf886ef0544375bf4axf 4
7ed9907193c0b09eb2b145dr187e05f 3
21e6f616308470a9a183b90b0855dfb3
20009e9ed0ce7f844813857e-cb9301qb
68a5d025cc8fe073736a087006c82896
edc110b315630713742drfa060e4a5b
bed60849e4ab0e9392a237401434a7a7
76413c7ba0a6fad743767f6d94x9aae7
8b84870f9eaa00f0001b5e44701b590
d94ae5791e305005230616c49487d240
21124b12f1f59d65adff800c0e8162c3

```

2,259,724

39800/s

113

```

dbe11023eeb94b7a436193c6c73b60be
0be06ea350976600eb93210563a5f39b
0f1bd311bb6376390c8cc10b20ccdafd
dfc0a1de21248565a6838e8f5ce435f4
dfc37eff05e9ca82c485c523ba4b201e
dbe1f2552633814af7441c7cfcff0dce
dfc0abd37538a7f3035f10e704311091
0f1ae471537e81fea974e61c20ae0c91
0f0097a0c3f2542c1047a092b1fdb66f
dfc207323fcb7c7e8b88320fc2507b18

```

sandsifter

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram. The lines are vertical and horizontal, with some diagonal connections, and the circles are of varying sizes, some acting as nodes or junctions.

& (Demo)

sandsifter

- 
- A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.
- & Hidden instructions
 - & Ubiquitous software bugs
 - & Hypervisor flaws
 - & Hardware bugs

Results

- ⌘ 0f0dxx
 - ⌘ Undocumented for non-/1 reg fields
- ⌘ 0f18xx, 0f{1a-1f}xx
 - ⌘ Undocumented until December 2016
- ⌘ 0fae{e9-ef, f1-f7, f9-ff}
 - ⌘ Undocumented for non-0 r/m fields until June 2014
- ⌘ dbe0, dbe1
- ⌘ df{c0-c7}
- ⌘ f1
- ⌘ {c0-c1}{30-37, 70-77, b0-b7, f0-f7}
- ⌘ {d0-d1}{30-37, 70-77, b0-b7, f0-f7}
- ⌘ {d2-d3}{30-37, 70-77, b0-b7, f0-f7}
- ⌘ f6 /1, f7 /1

Hidden instructions



& Catch:

- ✗ Undocumented instructions recognized by the disassembler are not found

Hidden instructions

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& Issue:

- ✗ Our “ground truth” (the disassembler) is also prone to errors

Software bugs

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& Every disassembler we tried as the
“ground truth” was littered with bugs.

Software bugs

- 
- & Most bugs only appear in a few tools,
and are not especially interesting
 - & Some bugs appeared in *all* tools
 - ⌘ These can be used to an attacker's advantage.

Software bugs



& 66e9xxxxxxxx (jmp)
& 66e8xxxxxxxx (call)

Software bugs



& 66 jmp

& Demo:

⌘ IDA

⌘ Visual Studio

⌘ objdump

⌘ QEMU

Software bugs

- 
- ⌘ 66 jmp
 - ⌘ Why does everyone get this wrong?
 - ⌘ AMD designed the 64 bit architecture
 - ⌘ Intel adopted... most of it.

Software bugs

- 
- ⌘ Issues when we can't agree on a standard
 - ⌘ sysret bugs
 - ⌘ Either Intel or AMD is going to be vulnerable when there is a difference
 - ⌘ Complex architecture
 - ⌘ Tools cannot parse a jump instruction

Software bugs



& Azure

⌘ CPUID / Trap flag bug

Hypervisor bugs



& Intel:

⌘ f00f bug on Pentium

& AMD:

⌘ Incorrect signals during decode

& Transmeta:

⌘ Of{71,72,73}xxxx

⌘ Premature #GPO signal during decode

Hardware bugs



& Our processors are not doing what we think they are

- ⌘ We need formal specifications
- ⌘ We need auditing tools
- ⌘ This is a start.

Conclusions

A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.

& Sandsifter lets us introspect what is otherwise a black box

Conclusions



& Open sourced:

- ⌘ The sandsifter scanning tool

- ⌘ github.com/xoreaxeaxeax/sandsifter

Conclusions

- 
- A decorative graphic on the left side of the slide, consisting of a network of thin, light-gray lines and small circles, resembling a circuit board or a neural network diagram.
- & Use sandsifter to audit your processor
 - & Reveal the instructions it really supports
 - & Search for hardware errata
 - & Break disassemblers,
emulators, and hypervisors
 - & Send us the results

Conclusions



&github.com/xoreaxeaxeax

⌘ sandsifter

⌘ M/o/Vfuscator

⌘ REpsych

⌘ x86 0-day PoC

⌘ Etc.

&Feedback? Ideas?

&domas

⌘ @xoreaxeaxeax

⌘ xoreaxeaxeax@gmail.com

